Property Search Prototype

Technical design document

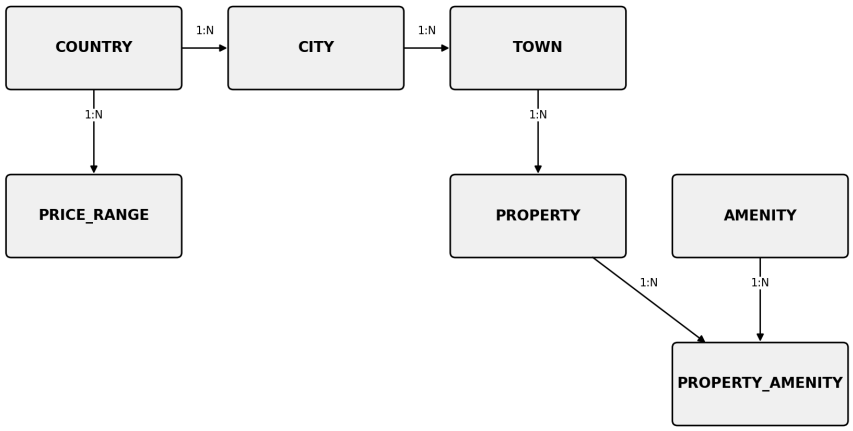
1. Architecture

The prototype follows a standard three-layer web architecture: a frontend, a backend API, and a relational database. This separation keeps the search logic on the server, where filters are validated and combined, while the frontend is only responsible for capturing input and rendering results.

- Frontend: a single-page application handling the tabs (For Sale / To Rent), the location and price dropdowns, the filter panel, and the results grid. Built responsively so the same codebase serves desktop and mobile.
- Backend API: exposes endpoints for populating dropdown options (locations, price ranges) and for running property searches. All filter combination logic lives here, not in the frontend.
- Database: a relational database (PostgreSQL) storing reference data (countries, cities, towns, price ranges, amenities) and the property listings themselves.

2. Data model

The schema separates reference data (things that rarely change: locations, price bands, amenity types) from transactional data (the property listings). This keeps dropdown population fast and independent of listing volume, and keeps the search query itself simple.



country

Field	Type	Notes
`id`	int, PK	Primary key
`name`	varchar	Kenya, Uganda, Tanzania
`currency_code`	varchar(3)	ISO code, e.g. KES, UGX, TZS

city

Field	Type	Notes
`id`	int, PK	Primary key
`country_id`	int, FK	References country.id
`name`	varchar	e.g. Nairobi, Kampala

town

Field	Type	Notes
id	int, PK	Primary key
city_id	int, FK	References city.id
name	varchar	e.g. Westlands, Kololo

price_range

Field	Type	Notes
id	int, PK	Primary key
country_id	int, FK	References country.id
listing_type	enum	'sale' or 'rent'
currency_code	varchar(3)	Matches country currency
min_price	bigint	Lower bound offered in the dropdown
max_price	bigint	Upper bound offered in the dropdown

property

Field	Type	Notes
id	int, PK	Primary key
town_id	int, FK	References town.id
listing_type	enum	'sale' or 'rent'
property_type	varchar	House, Apartment, Office, Land, etc.
price	bigint	In the local currency of its country
bedrooms	smallint	
bathrooms	smallint	
status	enum	active / inactive, for soft removal

amenity

Field	Type	Notes
id	int, PK	Primary key
name	varchar	e.g. Garden, Pool, Parking

property_amenity (join table)

Field	Type	Notes
property_id	int, FK	References property.id
amenity_id	int, FK	References amenity.id

3. Search query construction

A search request arrives as a set of optional parameters (country, listing_type, city, town, min_price, max_price, property_type, bedrooms, bathrooms, amenities). The backend builds a single query by appending a condition for each parameter that is present, and skipping any that are not. This is what allows filters to work individually or in any combination without special-case code per filter.

```
SELECT p.* FROM property p
  JOIN town t ON p.town_id = t.id
  JOIN city c ON t.city_id = c.id
  JOIN country co ON c.country_id = co.id
WHERE co.name = :country
  AND p.listing_type = :listing_type
  [AND p.price BETWEEN :min_price AND :max_price]
  [AND t.name = :town]
  [AND p.property_type = :property_type]
  [AND p.bedrooms >= :bedrooms]
  [AND p.bathrooms >= :bathrooms]
```

Bracketed clauses are appended only when the corresponding filter is set. Amenity filters (must have Garden AND Pool, for example) are applied as an additional join against property_amenity, or evaluated as a subquery requiring a matching row for each selected amenity, so that a property only qualifies when it satisfies all of them, not just one.

No ranking, scoring, or relevance calculation is involved. A property either satisfies every active condition or it is excluded. Result ordering (e.g. newest first, price ascending) is a simple ORDER BY and does not affect which rows match.

For the 'no matching properties' case, the API returns an empty result set with a 200 status rather than an error, and the frontend renders a clear empty state instead of a blank screen.

4. Indexing and performance

Reference tables (country, city, town, price_range, amenity) are small and can be cached in memory after first load, so they do not add query load. The property table is indexed to match common filter combinations:

- Composite index on (town_id, listing_type, price) to support the most frequent filter shape.
- Separate indexes on property_type and bedrooms for filters used less consistently together.
- A foreign key index on property_amenity(property_id, amenity_id) to keep amenity joins efficient.

This indexing strategy is standard relational database practice and requires no specialised search infrastructure. A dedicated search engine (such as Elasticsearch) would only become relevant if free-text search or relevance ranking is introduced later, which is outside the current requirements.

5. API surface (prototype scope)

- GET /countries — list of countries.
- GET /cities?country_id= — cities for a given country.
- GET /towns?city_id= — towns for a given city.
- GET /price-range?country_id=&listing_type= — currency and min/max bounds for the price dropdowns.
- GET /properties?[filters] — the main search endpoint, accepting all optional filter parameters described in section 3.

6. Tech stack summary

- Frontend: React, built responsive-first.
- Backend: REST API (Django, Node/Express or an equivalent framework).
- Database: PostgreSQL.
- Seed data: a fictional dataset spanning all three countries, both listing types, multiple property types, and varied amenities, used to validate every filter and filter combination.